

TINYML PLANT DETECTOR

Asaf Iron-Jobes – asafij@uw.edu – 2262052

Problem Statement: I want to build a ultra-low power machine learning powered plant detector that can classify plants based on camera input data. This will be useful for applications such as identifying poisonous plants in nature or just for enthusiasts to use to enjoy nature. Variations of this idea could also be used in a large network for agriculture applications such as identifying invasive species of weeds in real time or to identify features of crops such as disease or improved health. In all these cases they benefit from machine learning on the edge because of either low power constraints or unreliable access to fast network speeds.

Data set: I am planning to use this [large data set](#). The name of the data set is PlantNet-300K. It has over 1000 plant types and is over 32 Gb large so it should be sufficient for training. One benefit of this data set is that it has photos in various environments and backgrounds. For example, it has pictures of flowers in people's hands and also pictures of larger trees. My hope is that it contains enough of the types of plants in the pacific northwest, so the data set applies well to plants here for testing. I may use a k-fold test split strategy which I learned in my machine learning class EE443 but this causes more time for training so it may not be the best in which case I do a 80-20 split.

Modeling approach: My input is going to be camera data in real time. I am going to use a CNN with a MLP. I will use quantization and pruning but I may also be able to use distillation by using one of the plant identifying apps for mobile phones which use a much larger neural network architecture to distill the outputs. That may be hard if I cannot find an open-source version. Although it may change my initial plan is to use MobileNetV2. The MLP output final layer will have N output nodes for each plant in the final dataset after I remove irrelevant plants to Seattle's ecosystem.

Deployment plan: I am going to use the Arduino Nano BLE alongside the Arduino camera to collect camera data. I will also add an LCD screen to display the name of the plant if the confidence is above a certain threshold, otherwise it will say "no plant detected" or something similar. I will evaluate it by testing it on the plants in my neighborhood and my house plants in well-lit conditions to give it the best chance of success. One thing I will need to watch out for is the inference time per image. I need to make sure that my frame rate is less than this value in order to have real time signal processing.

Challenges & Solutions: One bottleneck could be that because the dataset is very large, it will take a long time to train which could become difficult if I need to iterate the design a lot. A solution to this could be to remove plant species that are not in the pacific northwest or Seattle. This could dramatically reduce the size of the data and therefore the training time. Another issue could be that the type of data and features are too complex for how many species there are and to get the model small enough to fit on the Nano it will lose a lot of the complexity that it needs to be effective. Another challenge could be figuring out how to diagnose underperformance from the model. Because it is looking for many plant types from real time camera data it will be hard to figure out the best way to improve the model

from just using the device and pointing the camera at plants. One way to get better data could be to take pictures and input them to the model in python and then evaluate the output. This way I can more concretely get the output data and use it to find the shortcoming of the model. One way to reduce the training time could be to use a transfer learning strategy which would essentially mean fine tuning a pretrained model rather than training from scratch but for the sake of the final project I will likely elect to train my own model from scratch to learn more. I plan to collect a smaller set of images of plants in the final data set from my actual neighborhood as a final way to fine tune my model to work better in the environments I will be testing it in.